

```

* Based on number of argument (nr) required for system call
* nr will be appended and macro will be expanded
* internal_syscall0, internal_syscall1 .... internal_syscall7
* for openat is 4 arguments system call hence macro will
expanded to
* internal_syscall4
*/

```

```

#undef internal_syscall4
#define internal_syscall4(number, err, arg1, arg2, arg3,
arg4) \
({ \

unsigned long int resultvar;
TYPEFY (arg4, __arg4) = ARGIFY (arg4) \
TYPEFY (arg3, __arg3) = ARGIFY (arg3); \
TYPEFY (arg2, __arg2) = ARGIFY (arg2); \
TYPEFY (arg1, __arg1) = ARGIFY (arg1); \
register TYPEFY (arg4, _a4) asm ("r10") = __arg4; \
register TYPEFY (arg3, _a3) asm ("rdx") = __arg3; \
register TYPEFY (arg2, _a2) asm ("rsi") = __arg2; \
register TYPEFY (arg1, _a1) asm ("rdi") = __arg1; \

/* Below asm code snippet, system call number and arguments
are loaded
* and syscall instruction is executed which ultimately
changes the
* mode to kernel space and leads to invocation of kernel
space system call handler
*/

asm volatile ( \
"syscall\n\t" \
: "=a" (resultvar) \
: "0" (number), "r" (_a1), "r" (_a2), "r" (_a3), "r" (_
a4) \
: "memory", REGISTERS_CLOBBED_BY_SYSCALL); \
(long int) resultvar; \
})

```

So now we understand how the glibc system call wrapper prepares the arguments and executes *syscall* instruction to change the mode from user space to kernel space. We can see that the system call wrapper hides all architecture level assembly complexity related with argument preparation and execution of *syscall* instructions from the user space application program. For invoking a particular system call, the user space application only needs to invoke the wrapper function provided by glibc like a normal C function or API. There is no need to bother about the underlying architecture related to assembly code/instructions.

Calling system call with *syscall(2)* interface

We have seen how glibc wrapper hides the system call invocation complexity from the user space application program. But what if the glibc wrapper doesn't exist for some of the system calls (such as *futex(2)*). In order to handle or invoke such system calls from user space applications, glibc provides a generic library function called *syscall(2)*. This *syscall(2)* function is implemented as assembly in glibc (as it needs to take care of architecture-specific assembly instructions). The implementation of *syscall(2)* function in glibc for x86-64 is given below. Please check the comments marked in red.

```

#include <sysdep.h>

/* Please consult the file sysdeps/unix/sysv/linux/x86-64/
sysdep.h for more information about the value -4095 used
below. */

/* Usage: long syscall (syscall_number, arg1, arg2, arg3,
arg4, arg5, arg6) We need to do some arg shifting, the
syscall_number will be rax. */

/* syscall function is moving the argument values to the
registers
* and then calling the assembly instruction syscall
* syscall is a user space glibc library function, first the
* arguments will be in the registers used for calling user
space
* function. Therefore, before invoking syscall instruction
function
* is doing argument shifting as per kernel specification
*/

.text
ENTRY (syscall)
    movq %rdi, %rax /* Syscall number -> rax. */
    movq %rsi, %rdi /* shift arg1 - arg5. */
    movq %rdx, %rsi
    movq %rcx, %rdx
    movq %r8, %r10
    movq %r9, %r8
    movq 8(%rsp), %r9 /* arg6 is on the stack. */
    syscall /* Do the system call. */
    cmpq $-4095, %rax /* Check %rax for error. */
    jae SYSCALL_ERROR_LABEL /* Jump to error handler if
error. */
    ret /* Return to caller. */

```

PSEUDO_END (syscall)

From the man page of *syscall(2)*

```

#include <sys/syscall.h> /* Definition of SYS_* constants */
#include <unistd.h>

```